Module 4 JavaScript & jQuery

JavaScript

- For a Web page, **HTML** supplies document **content and structure** while **CSS** provides **presentation styling.**
- In addition, client-side scripts can **control browser actions** associated with a Web page.
- Client-side scripts are almost written in the **Javascript** language to control browser's actions.
- Client-side scripting can make Web pages more **dynamic** and more **responsive**

- **What can JavaScript do?**
- JavaScript can dynamically modify an HTML page.
- JavaScript can validate user input.
- Javascript is a full-featured programming language.
- JavaScript can be used to create cookies
- JavaScript user interaction does not require any communication with the server.

Benefits of learning JavaScript:

Web Development	Desktop App Development	Mobile App Development
• jQuery • Angular Js • React Js • VueJS • NodeJS	• Electron JS	• Angular Js • React Js • VueJS • React Native • NodeJS

Software Requirement for JavaScript :

- HTML Editor
- Web Browser

Editor for JavaScript :

- Notepad
- Notepad++
- VS Code
- Sublime
- Atom

Web Browser for JavaScript :

- Google Chrome
- Mozilla Firebox
- Internet Explorer
- Safari

Advantages:

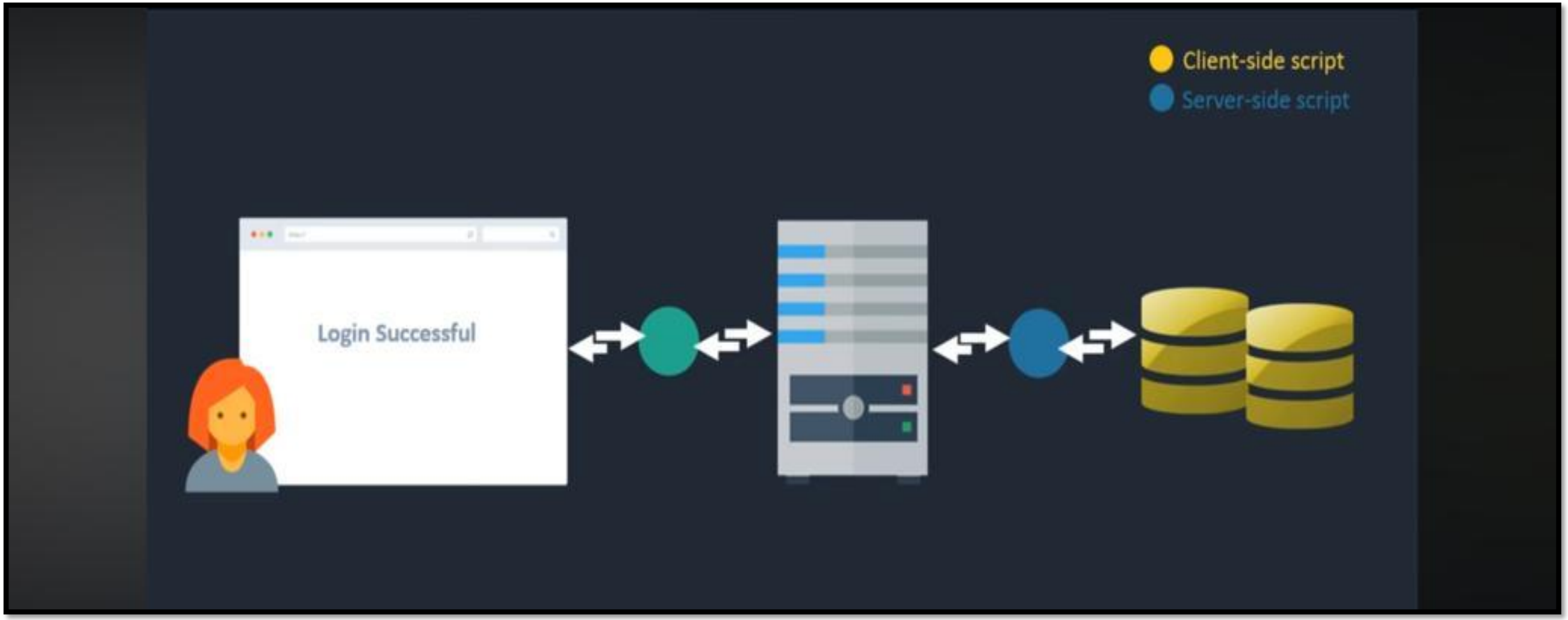
- **Speed**. Client-side JavaScript is **very fast because it can be run immediately within the client-side browser**. Unless outside resources are required, JavaScript is unhindered by network calls to a backend server.
- **Simplicity**. JavaScript is relatively simple to learn and implement.
- **Popularity**. JavaScript is used everywhere on the web.
- Gives the ability to create rich interfaces.

Disadvantages:

- **Client-Side Security.** Because the code executes on the users' computer, in some cases it can be exploited for malicious purposes. This is one reason some people choose to disable Javascript.
- **Browser Support.** JavaScript is sometimes interpreted differently by different browsers. This makes it somewhat difficult to write cross-browser code.

Difference

1. Server-side scripting is utilized in the backend when the source code is invisible or hidden on the client side. In contrast, client-side scripting is utilized at the front end, which users may access via the browser.
2. The files are inaccessible to the client-side script. In contrast, the server-side script has access to the files on the web server.
3. Server-side scripting is very useful for personalizing web pages and implementing dynamic updates to websites. In contrast, the client-side script may effectively minimize the load on the server.
4. The client-side script responds faster than the server-side script.
5. Client-side scripting is less secure than server-side scripting due to the server-side scripts are commonly concealed from the client end. In contrast, the client-side scripts are exposed to users.
6. The client-side script is run on a local computer. In contrast, the server-side script is run on a remote machine.
7. The client-side programming languages are ***HTML, CSS, and JavaScript***. In contrast, server-side scripting programming languages, including ***PHP, ColdFusion, Python, ASP.net, Java, C++, Ruby, C#,***
8. A server-side script communicates with the server when it is executed. In contrast, Client-side scripting doesn't need server involvement.
9. The script on the client side lacks security. However, the server-side script is protected.
10. The client-side script is dependent on the user's browser version. In contrast, the Server-side doesn't depend on the client.


Client-side script
Server-side script
Login Successful

Structure of JavaScript

- JavaScript can be implemented using JavaScript statements that are placed within the <script>…</script>HTML tags in a web page.
- You can place the <script> tag tags, containing your JavaScript, anywhere within you web page, but it is normally recommended that you should keep it within the <head> tags.
- **JavaScript syntax will look as follows:**

```
<script language =”javascript” type=”text/javascript”>
        //javascript code here
</script>
```

Code

```
<html>
 <head>
  <title>HTML script Tag</title>
 </head>
 <body>
  <script>
   // Java Script Code Here
  </script>
 </body>
</html>
```

Example: Internal JavaScript

```
<html>
    <body>
        <script>
            document.write("Hello JavaScript by JavaScript");
        </script>
    </body>
</html>
```

Output:



Example: External JavaScript

- An external JavaScript file must be saved by **.js** extension. It is recommended to embed all JavaScript files into a single file. It increases the speed of the webpage.
- **Example:** create an external JavaScript file that prints Hello Javatpoint in a alert dialog box.

index.html

```
<html>
<head>
<script type="text/javascript" src="message.js"></script> </head>
<body>
<p>Welcome to JavaScript</p>
<form>
<input type="button" value="click" onclick="msg()"/>
</form>
</body>
</html>
```

message.js

```
function msg() {  alert("Hello PPSU");
                  }
```

Output:

Hello PPSU

Comments

- The JavaScript comments are meaningful way to deliver message. It is used to add information about the code, warnings or suggestions so that end user can easily interpret the code.
- There are two types of comments in JavaScript.
    1. Single-line Comment
    2. Multi-line Comment

```
<script>
// It is single line comment
document.write("hello javascript");
</script>
```

Single Line Comment

```
<script>
/* It is multi line comment.
It will not be displayed */
document.write("example of javascript multiline comment");
</script>
```

Multi Line Comment

JavaScript Output

JavaScript can "display" data in different ways:

1. Writing into an HTML element, using innerHTML.

- To access an HTML element, JavaScript can use, the **document.getElementById(id)** method.
- The **id attribute defines the HTML element**. The **innerHTML property defines the HTML content** .

2. Writing into the HTML output using document.write().

- Using **document.write()** after an HTML document is loaded, will delete all existing HTML

3. Writing into an **alert box**, using window.alert().

- You can use an **alert box to display data**

4. Writing into the browser console, using console.log().

- For **debugging purposes**, you can call the **console.log()** method in the browser to display data.

1. Using innerHTML

- To access an HTML element, JavaScript can use the **document.getElementById(id) method**.
- The **id attribute defines the HTML element**. The **innerHTML property defines the HTML content**:

```
<html>
<body>
<h2>My First Web Page</h2>
<p>My First Paragraph.</p>
<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML = 5 + 6;
</script>
</body>
</html>
```



2. Using document.write()

- For testing purposes, it is convenient to use document.write():

```
<html>
<body>
<h2>My First Web Page</h2>
<p>My first paragraph.</p>
<p>Never call document.write after the document has finished loading.
It will overwrite the whole document.</p>
<script>
document.write(5 + 6);
</script>
</body>
</html>
```



```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page</h2>
<p>My first paragraph.</p>

<button type="button" onclick="document.write(5 + 6)">Try it</button>

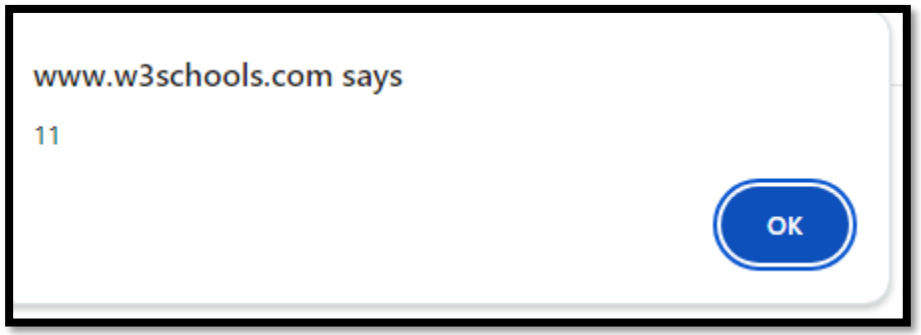
</body>
</html>
```



3. Using window.alert()

- You can use an alert box to display data:

```
<html>
<body>
<h2>My First Web Page</h2>
<p>My first paragraph.</p>
<script>
window.alert(5 + 6);
</script>
</body>
</html>
```



4. Using console.log()

- For **debugging purposes**, you can call the console.log() method in the browser to display data.

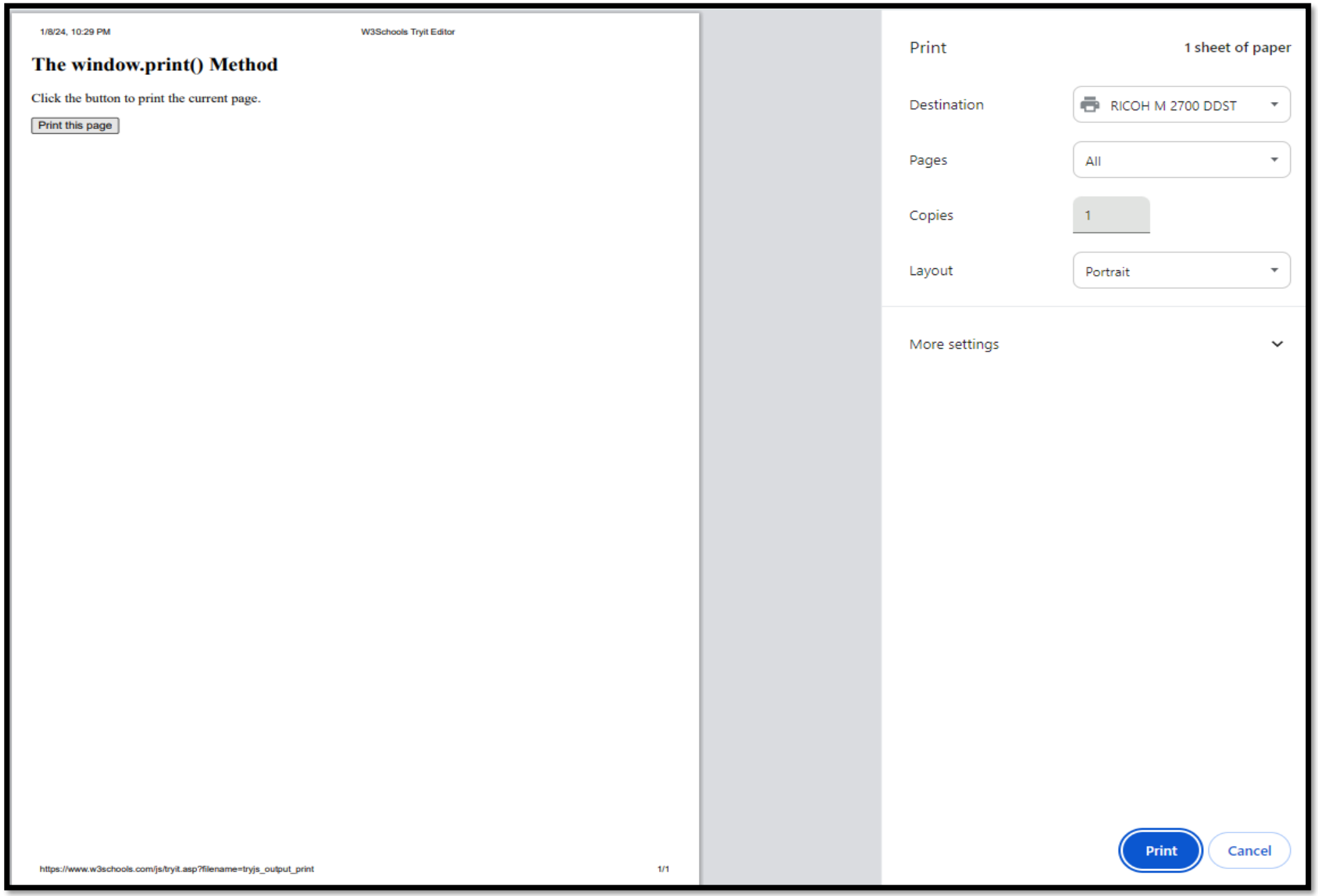
```
<html>
<body>
<h2>Activate Debugging</h2>
<p>F12 on your keybord will activate debugging.</p>
<p>Then select "Console" in the debugger menu.</p>
<p>Then click Run again.</p>
<script>
console.log(5 + 6);
</script>
</body>
</html>
```

5.JavaScript Print

- JavaScript does not have any print object or print methods.
- You cannot access output devices from JavaScript.
- The only exception is that you can call the **window.print() method** in the browser to print the content of the current window
- **Example:**

```
<html>
<body>
<button onclick="window.print()">
Print this page</button>
</body>
</html>
```




1/8/24, 10:29 PM
W3Schools Tryit Editor
The window.print() Method
Click the button to print the current page.
Print this page
https://www.w3schools.com/js/tryit.asp?filename=tryjs_output_print
1/1
Print
1 sheet of paper
Destination
RICOH M 2700 DDST
Pages
All
Copies
1
Layout
Portrait
More settings
Print
Cancel

Data Types

- One of the most fundamental characteristics of a programming language is the set of data types it supports. These are the type of values that can be represented and manipulated in a programming language.
- JavaScript allows you to work with three primitive data types:

Data Type	Description
String	represents sequence of characters e.g. "hello"
Number	represents numeric values e.g. 100
Boolean	represents boolean value either false or true
Undefined	represents undefined value
Null	represents null i.e. no value at all

JavaScript Variable

- Variables are containers for storing data (values).
- Declaring a variable tells Javascript that a variable of a given name exists so that the javascript interpreter can understand references to that variable name throughout the rest of the script.
- A **JavaScript variable** is simply a name of storage location. There are two types of variables in JavaScript : 1) local variable and 2) global variable.

Example:

```
1) x = 10;
2) var _value="sonoo";
3) let x = 10;
4) const PI = 3.141592653589793;
5) <script>
        var x = 10;
        var y = 20;
        var z=x+y;
        document.write(z);
   </script>
```

5. JavaScript Var

```
<html>
<body>
<h2>Redeclaring a Variable Using var</h2>
<p id="demo"></p>
<script>
var  x = 10;// Here x is 10
{
var x = 2;// Here x is 2
}
// Here x is 2
document.getElementById("demo").innerHTML = x;
</script>
</body>
</html>
```

Output:

Redeclaring a Variable Using var

2

4. JavaScript Let

- Variables defined with let cannot be Redeclared.
- Variables defined with let must be Declared before use.
- Variables defined with let have Block Scope.

```
<html>
<body>
<h2>Redeclaring a Variable Using let</h2>
<p id="demo"></p>
<script>
let  x = 10; // Here x is 10
{
  let x = 2;   // Here x is 2
}
// Here x is 10
document.getElementById("demo").innerHTML = x;
</script>
</body>
</html>
```

Output:

Redeclaring a Variable Using let

10

6. JavaScript Const

- **Const** is another keyword to declare a variable when you do not want to change the value of that variable for the whole program.
- The difference is just that var is for normal variable declaration whose value can be changed, whereas a variable value declared using const keyword cannot be changed.

```
<html>
<body>
<script>
    const x = 16;
    document.write("The value of const variable x = " + x”);
</script>
</body>
</html>
```

Output:

The value of const variable x = 16

1. JavaScript local variable

- A JavaScript local variable is declared inside block or function. It is accessible within the function or block only.
- For example:

```
<script>
function abc(){
var x=10;//local variable
}
</script>
```

Or,

```
<script>
If(10<13){
var y=20;//JavaScript local variable
}
</script>
```

2. JavaScript global variable

- A **JavaScript global variable** is accessible from any function. A variable i.e. declared outside the function or declared with window object is known as global variable.
- For example:

```
<html>
<body>
<script>
var data=200;//gloabal variable
function a(){
document.writeln(data);
}
function b(){
document.writeln(data);
}
a();//calling JavaScript function
b();

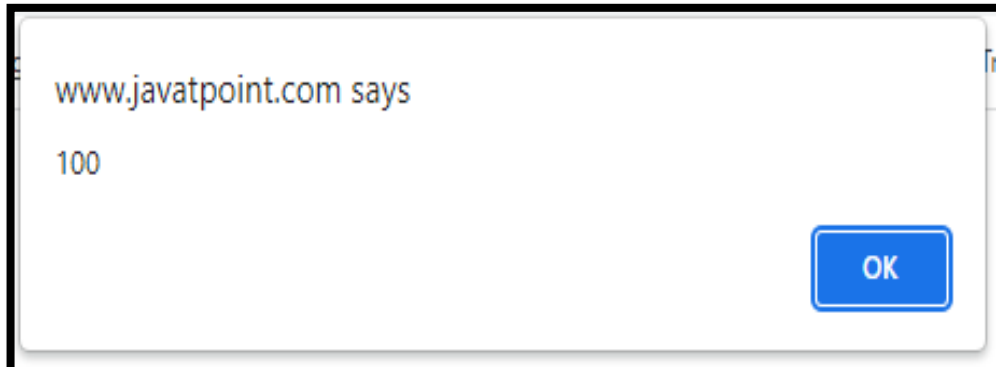
</script>
</body>
</html>
```

```
200 200
```

3. Declaring JavaScript global variable within function

- To declare JavaScript global variables inside function, you need to use **window object**.
- For example:
- Now it can be declared inside any function and can be accessed from any function. For example:

```
<html>
<body>
<script>
function m(){
window.value=100;//declaring global variable by window object
}
function n(){
alert(window.value);//accessing global variable from other function
}
m();
n();
</script>
</body>
</html>
```



When to Use var, let, or const?

1.Always declare variables
2. Always use const if the value should not be changed
3. Always use const if the type should not be changed (Arrays and Objects)
4. Only use let if you can't use const
5. Only use var if you MUST support old browsers

- Difference Between var, let and const

	Scope	Redeclare	Reassign	Hoisted	Binds this
var	No	Yes	Yes	Yes	Yes
let	Yes	No	Yes	No	No
const	Yes	No	No	No	No

Operator

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. String Operators
5. Logical Operators
6. Bitwise Operators
7. Ternary Operators
8. Type Operators

1. Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

2. Assignment Operators

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

3. Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

4. String Operators

- All the comparison operators above can also be used on strings:

```
let text1 = "A";
let text2 = "B";
let result = text1 < text2;
```

- strings are compared alphabetically
- The + can also be used to add (concatenate) strings
- The += assignment operator can also be used to add (concatenate) strings

```
let text1 = "What a very ";
text1 += "nice day";
```

5. Logical Operators

Operator	Description
&&	logical and
||	logical or
!	logical not

6. Type Operators

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

7. Bitwise Operators

- Bit operators work on 32 bits numbers.
- Any numeric operand in the operation is converted into a 32 bit number. The result is converted back to a JavaScript number.

Operator	Description	Example	Same as	Result	Decimal
&	AND	5 & 1	0101 & 0001	0001	1
|	OR	5 | 1	0101 | 0001	0101	5
~	NOT	~ 5	~0101	1010	10
^	XOR	5 ^ 1	0101 ^ 0001	0100	4
<<	left shift	5 << 1	0101 << 1	1010	10
>>	right shift	5 >> 1	0101 >> 1	0010	2
>>>	unsigned right shift	5 >>> 1	0101 >>> 1	0010	2

Strings

- A string can be defined as a sequence of letters, digits, punctuation and so on.
- A string in a JavaScript is wrapped with single or double quotes
- Strings can be joined together with the + operator, which is called concatenation.
  - For Example,
  - mystring = “my college name is ” + “Darshan”;
- As string is an object type it also has some useful features.
  - For Example,
  - lenStr = mystring.length;
  - Which returns the length of the string in integer

Strings (Cont.)

- There are also number of methods available for string.

Method	Description
charAt	Returns the character at a specific index
indexOf	Find the first index of a character
lastIndexOf	Find the last index of a character
substring / substr	Return a section of a string.
replace	Replaces a specified value with another value in a string
toLowerCase	Convert a string to lower case.
toUpperCase	Convert a string to upper case.

Strings (Cont.)

- An escape sequence is a sequence of characters that does not represent itself when used inside a character or string, but is translated into another character or a sequence of characters that may be difficult or impossible to represent directly.
- Some Useful Escape sequences :

Sequence	Character
\t	Tab
\n	Newline
\r	Carriage return
\"	Double Quote
\'	Single Quote
\\	Backslash

Functions

- A JavaScript function is a block of code designed to perform a particular task.
- A JavaScript function is executed when "something" invokes it.
- A JavaScript function is defined with the function keyword, followed by a name, followed by parentheses ().
- The parentheses may include parameter names separated by commas: (parameter1, parameter2, ...)
- The code to be executed, by the function, is placed inside curly brackets.
- Example :

Code

```
function myFunction(p1, p2) {
        return p1 * p2;
}
```

Functions

- When JavaScript reaches a return statement, the function will stop executing.
- If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.
- The code inside the function will execute when "something" invokes (calls) the function:
  - When an event occurs (when a user clicks a button)
  - When it is invoked (called) from JavaScript code
  - Automatically (self invoked)

```
function toCelsius(fahrenheit) {
  return (5/9) * (fahrenheit-32);
}

let value = toCelsius(77);
```

Functions Used as Variable Values

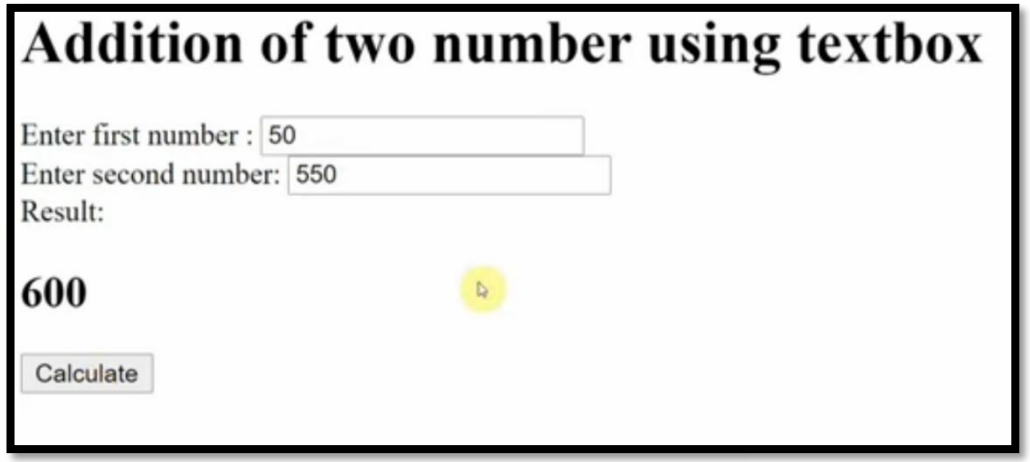
- Functions can be used the same way as you use variables, in all types of formulas, assignments, and calculations.

- let x = toCelsius(77);

  let text = "The temperature is " + x + " Celsius";

- let text = "The temperature is " + toCelsius(77) + " Celsius";

Example




```
1   <!DOCTYPE html>
2   <html>
3       <head>
4           <script>
5               function fun1(){
6                   var fnum=document.getElementById("txtf1").value;
7                   var snum=document.getElementById("txtf2").value;
8
9                   var tot= parseInt(fnum)  +  parseInt(snum);
10
11                  document.getElementById("result").innerHTML=tot;
12              }
13          </script>
14      </head>
15
16      <body>
17              <h1>Addition of two number using textbox</h1>
18              Enter first number : <input id="txtf1" /><br/>
19              Enter second number: <input id="txtf2" /> <br/>
20              Result: <h2 id="result"></h2>
21              <button onclick="fun1()">Calculate</button>
22      </body>
23  </html>
```

Javascript Conditional Statements

- In JavaScript we have the following conditional statements:
- Use if to specify a block of code to be executed, if a specified condition is true
- Use else to specify a block of code to be executed, if the same condition is false
- Use else if to specify a new condition to test, if the first condition is false
- Use switch to specify many alternative blocks of code to be executed

1. IF Satement

Use the if statement to specify a block of JavaScript code to be executed if a condition is true.

```
if(expression){
//content to be evaluated
}
```

Example:

```
<html>
<body>
<script>
var a=20;
if(a>10){
document.write("value of a is greater than 10");
}
</script>
</body>
</html>
```

2. IF…ELSE Satement

It evaluates the content whether condition is true of false. The syntax of JavaScript if-else statement is given below.

```
if(expression){
//content to be evaluated if condition is true
}
else{
//content to be evaluated if condition is false
}
```

Example:

```
<html>
<body>
<script>
var a=20;
if(a%2==0){
document.write("a is even number");
}
else{
document.write("a is odd number");
}
</script>
</body>
</html>
```

3. IF…ELSE IF Satement

It evaluates the content only if expression is true from several expressions. The signature of JavaScript if else if statement is given below.

```
if(expression1){
//content to be evaluated if expression1 is true
}
else if(expression2){
//content to be evaluated if expression2 is true
}
else if(expression3){
//content to be evaluated if expression3 is true
}
else{
//content to be evaluated if no expression is true
}
```

3. IF…ELSE IF Satement

Example:

```
<html>
<body>
<script>
var a=20;
if(a==10){
document.write("a is equal to 10");
}
else if(a==15){
document.write("a is equal to 15");
}
else if(a==20){
document.write("a is equal to 20");
}
else{
document.write("a is not equal to 10, 15 or 20");
}
</script>
</body>
</html>
```

JavaScript Loops

- The **JavaScript loops** are used *to iterate the piece of code* using for, while, do while or for-in loops. It makes the code compact. It is mostly used in array.
- There are four types of loops in JavaScript.
- 1. for loop
- 2. while loop
- 3. do-while loop
- 4. for-in loop

1. For loop

The **JavaScript for loop** *iterates the elements for the fixed number of times*. It should be used if number of iteration is known. The syntax of for loop is given below.

```
for (initialization; condition; increment)
{
code to be executed
}
```

- **Example:**

```
<!DOCTYPE html>
<html>
<body>
<script>
for (i=1; i<=5; i++)
{
document.write(i + "<br/>")
}
</script>
</body>
</html>
```

2. While loop

- The **JavaScript while loop** *iterates the elements for the infinite number of times*. It should be used if number of iteration is not known. The syntax of while loop is given below.

```
while (condition)
{
code to be executed
}
```

- **Example:**

```
<!DOCTYPE html>
<html>
<body>
<script>
var i=11;
while (i<=15)
{
document.write(i + "<br/>");
i++;
}
</script>
</body>
</html>
```

3. Do – while lopp

- The **JavaScript do while loop** *iterates the elements for the infinite number of times* like while loop. But, code is *executed at least* once whether condition is true or false. The syntax of do while loop is given below.

do{

code to be executed

}while (condition);

- **Example:**

```
<html>
<body>
<script>
var i=21;
do{
document.write(i + "<br/>");
i++;
}while (i<=25);
</script>
</body>
</html>
```

Arrays

JavaScript array is an object that represents a collection of similar type of elements.

There are 3 ways to construct array in JavaScript

1. By array literal
2. By creating instance of Array directly (using new keyword)
3. By using an Array constructor (using new keyword)

1) JavaScript array literal

The syntax of creating array using array literal is given below:

```
var arrayname=[value1,value2.....valueN];
```

```
<html>
<body>
<script>
var emp=["Sonoo","Vimal","Ratan"];
for (i=0;i<emp.length;i++){
document.write(emp[i] + "<br/>");
}
</script>
</body>
</html>
```

Sonoo
Vimal
Ratan

2) JavaScript Array directly (new keyword)

The syntax of creating array directly is given below:

```
var arrayname=new Array();
```

Here, **new keyword** is used to create instance of array.

```
<html>
<body>
<script>
var i;
var emp = new Array();
emp[0] = "Arun";
emp[1] = "Varun";
emp[2] = "John";

for (i=0;i<emp.length;i++){
document.write(emp[i] + "<br>");
}
</script>
</body>
</html>
```

Arun
Varun
John

3) JavaScript array constructor (new keyword)

Here, you need to create instance of array by passing arguments in constructor so that we don't have to provide value explicitly.

The example of creating object by array constructor is given below.

```
<script>
var emp=new Array("Jai","Vijay","Smith");
for (i=0;i<emp.length;i++){
document.write(emp[i] + "<br>");
}
</script>
```

Test it Now

Output of the above example

```
Jai
Vijay
Smith
```

Pop up Boxes

- Popup boxes can be used to raise an alert, or to get confirmation on any input or to have a kind of input from the users.
- JavaScript supports **three** types of popup boxes.
  1. Alert box
  2. Confirm box
  3. Prompt box

1. Alert box

- An **alert box** is used if you want to **make sure** information **comes through** to the **user**.
- When an alert box pops up, the user will **have to click "OK"** to proceed.
- It can be used to display the result of validation.

Code

```
<html>
   <head>
            <title>Alert Box</title>
   </head>
   <body>
            <script>
                     alert("Hello World");
            </script>
   </body>
</html>
```

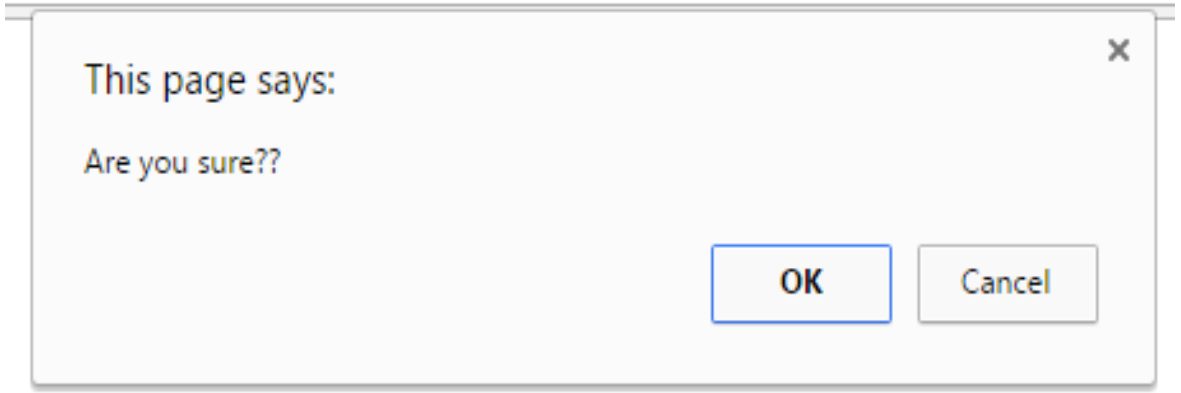


2. Confirm Box

- A **confirm box** is used if you want the user to **accept something**.
- **When** a confirm box **pops up**, the user will have to **click** either "**OK**" or "**Cancel**" to proceed, If the user clicks "**OK**", the box **returns true**. If the user clicks "**Cancel**", the box **returns false**.
- Example :

Code

```
<script>
  var a = confirm(“Are you sure??");
  if(a==true) {
            alert(“User Accepted”);
  }
  else   {
            alert(“User Cancled”);
  }
</script>
```

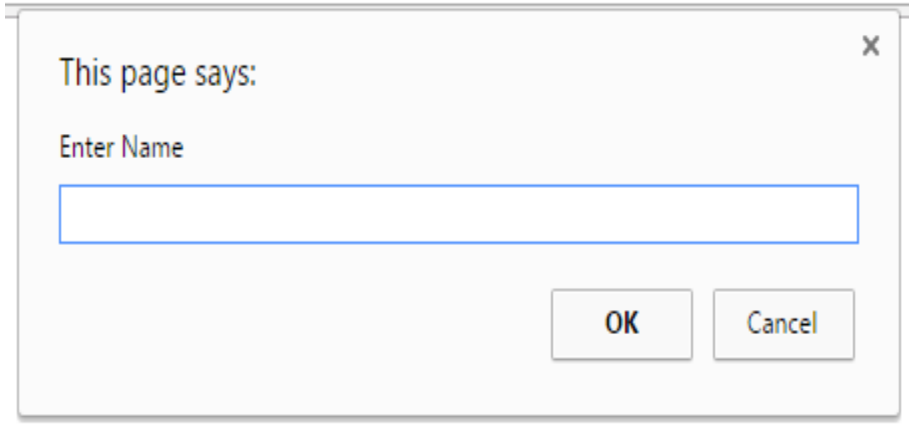


3. Prompt Box

- A **prompt box** is used if you want the user to **input a value**.
- **When** a prompt box **pops up**, user have to click either "**OK**" or "**Cancel**" to proceed, If the user clicks "**OK**" the box **returns the input value**, If the user clicks "**Cancel**" the box **returns null**.

Code

```
<script>
  var a = prompt(“Enter Name");
  alert(“User Entered ” + a);
</script>
```



JavaScript Objects

- A javaScript object is **an entity having state and behavior (properties and method)**. For example: car, pen, bike, chair, glass, keyboard, monitor etc.
- JavaScript is an object-based language. Everything is an object in JavaScript.

❑Creating Objects in JavaScript

- There are 3 ways to create objects.
  - By object literal
  - By creating instance of Object directly (using new keyword)
  - By using an object constructor (using new keyword)

1) JavaScript Object by object literal

The syntax of creating object using object literal is given below:

object={property1:value1,property2:value2.....propertyN:valueN}

```
<html>
<body>
<script>
emp={id:102,name:"Shyam Kumar",salary:40000}
document.write(emp.id+" "+emp.name+" "+emp.salary);
</script>
</body>
</html>
```

102 Shyam Kumar 40000

2) By creating instance of Object

The syntax of creating object directly is given below:

var objectname=new Object();

Here, **new keyword** is used to create object.

```
<html>
<body>
<script>
var emp=new Object();
emp.id=101;
emp.name="Ravi Malik";
emp.salary=50000;
document.write(emp.id+" "+emp.name+" "+emp.salary);
</script>
</body>
</html>
```

101 Ravi Malik 50000

3) By using an Object constructor

- Create function with arguments.
- Each argument value can be assigned in the current object by using **this keyword**.
- The **this keyword** refers to the current object.

```
<html>
<body>
<script>
function emp(id,name,salary){
this.id=id;
this.name=name;
this.salary=salary;
}
e=new emp(103,"Vimal Jaiswal",30000);

document.write(e.id+" "+e.name+" "+e.salary);
</script>
</body>
</html>
```

```
103 Vimal Jaiswal 30000
```

JavaScript’s inbuilt Objects

- JavaScript comes with some inbuilt objects which are,
  - String
  - Date
  - Array
  - Boolean
  - Math
  - RegExp

    etc….

Math Object in JavaScript

- The Math object allows you to perform mathematical tasks.
- The Math object includes several mathematical constants and methods.
- Example for using properties/methods of Math:

Code

```
<script>
        var x=Math.PI;
        var y=Math.sqrt(16);
</script>
```

Math Object in JavaScript

- Math object has some properties which are,

Properties	Description
E	Returns Euler's number(approx.2.718)
LN2	Returns the natural logarithm of 2 (approx.0.693)
LN10	Returns the natural logarithm of 10 (approx.2.302)
LOG2E	Returns the base-2 logarithm of E (approx.1.442)
LOG10E	Returns the base-10 logarithm of E (approx.0.434)
PI	Returns PI(approx.3.14)
SQRT1_2	Returns square root of ½
SQRT2	Returns square root of 2

Math Object in JavaScript

- Math object has some properties which are,

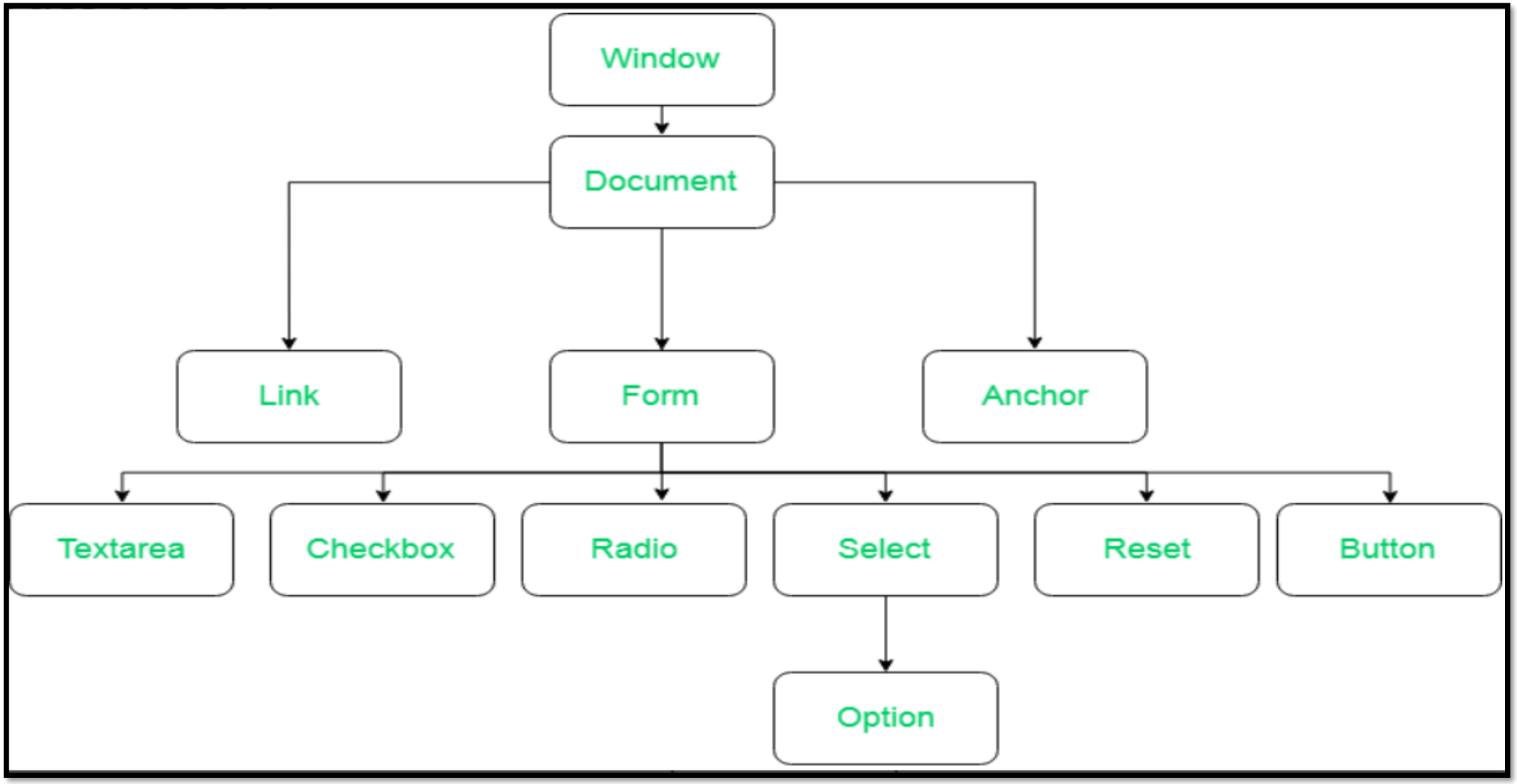
Method	Description
abs(x)	Returns the absolute value of x
sin(x)	Returns the sine of x (x is in radians)
cos(x)	Returns the cosine of x (x is in radians)
tan(x)	Returns the tan of x (x is in radians)
acos(x)	Returns the arccosine of x, in radians
asin(x)	Returns the arcsine of x, in radians
atan(x)	Returns the arctangent of x as a numeric value
atan2(x)	Returns arctangent of x
random(x)	Returns random floating number between 0 to 1

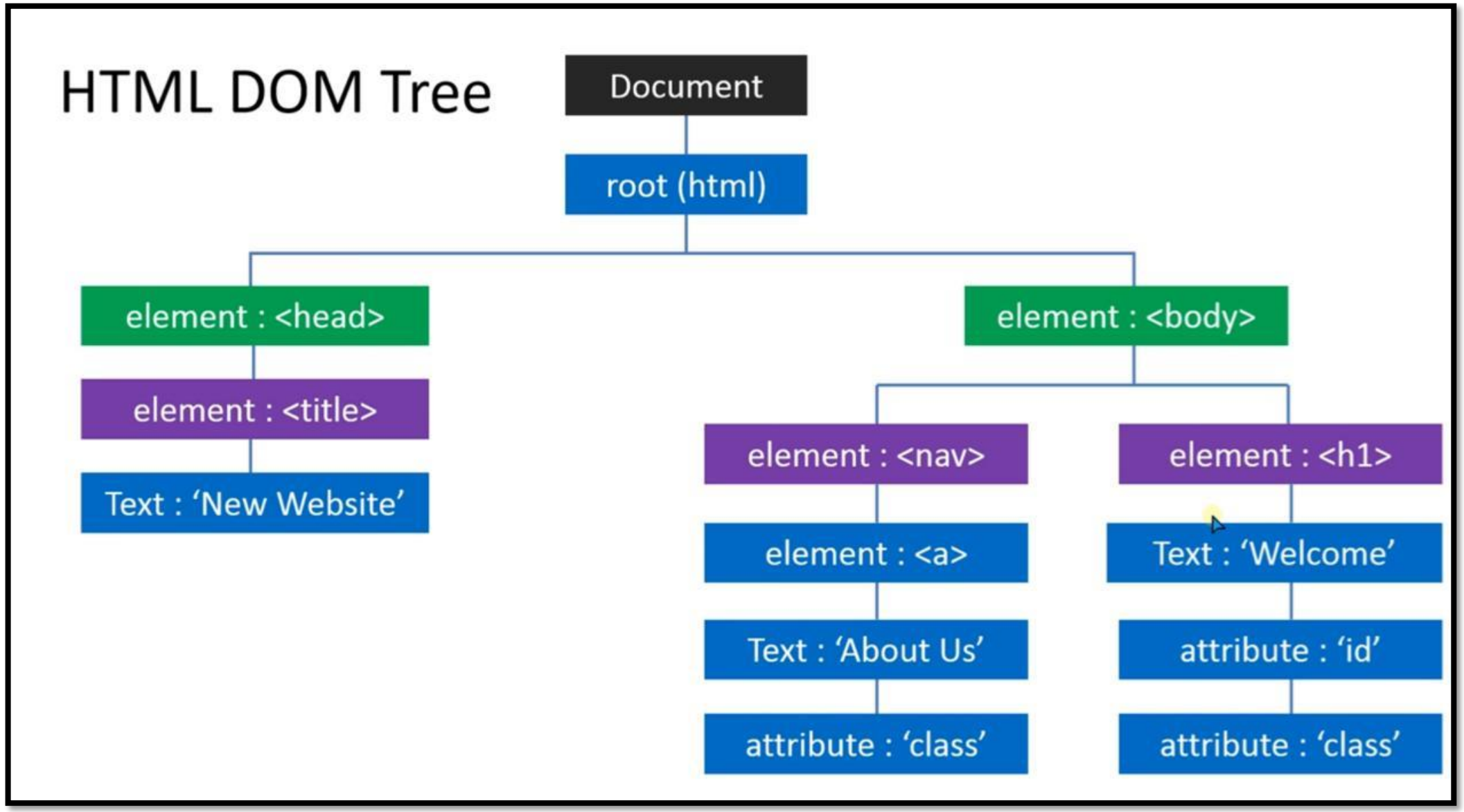
Method	Description
exp(x)	Returns the value of Ex
ceil(x)	Returns x, rounded upwards to the nearest integer
floor(x)	Returns x, rounded downwards to the nearest integer
log(x)	Returns the natural logarithm(base E) of x
round(x)	Rounds x to the nearest integer
pow(x,y)	Returns the value of x to the power of y
max(x,y,z,...,n)	Returns the number with the highest value
sqrt(x)	Returns the square root of x

Document Object Model

- The **document object** represents the whole html document.
- When html document is loaded in the browser, it becomes a document object.
- It is the root element that represents the html document.
- It has properties and methods.
- By the help of document object, we can add dynamic content to our web page.

Properties of document object




HTML DOM Tree
Document
root (html)
element : <head>
element : <title>
Text : 'New Website'
element : <body>
element : <nav>
element : <a>
Text : 'About Us'
attribute : 'class'
element : <h1>
Text : 'Welcome'
attribute : 'id'
attribute : 'class'

Methods of document object

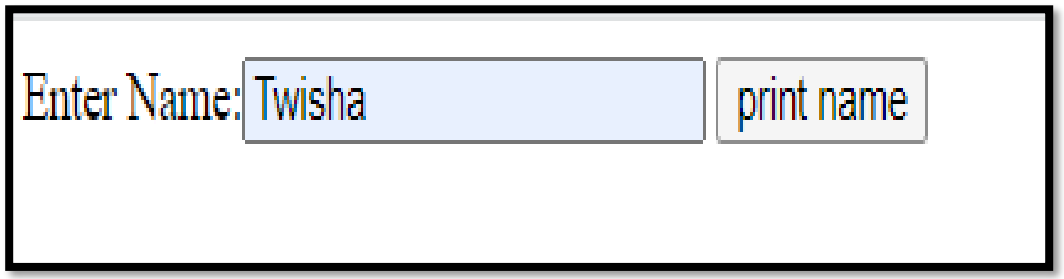
Method	Description
write("string")	writes the given string on the doucment.
writeln("string")	writes the given string on the doucment with newline character at the end.
getElementById()	returns the element having the given id value.
getElementsByName()	returns all the elements having the given name value.
getElementsByTagName()	returns all the elements having the given tag name.
getElementsByClassName()	returns all the elements having the given class name.

Accessing field value by document object

- **document.form1.name.value** is used to get the value of name field.
- Here, **document** is the root element that represents the html document.
- **form1** is the name of the form.
- **name** is the attribute name of the input text.
- **value** is the property, that returns the value of the input text.

```
<script type="text/javascript">
function printvalue()
{
var name=document.form1.name.value;
alert("Welcome: "+name);
}
</script>

<form name="form1">
Enter Name:<input type="text" name="name"/>
<input type="button" onclick="printvalue()" value="print name"/>
</form>
```

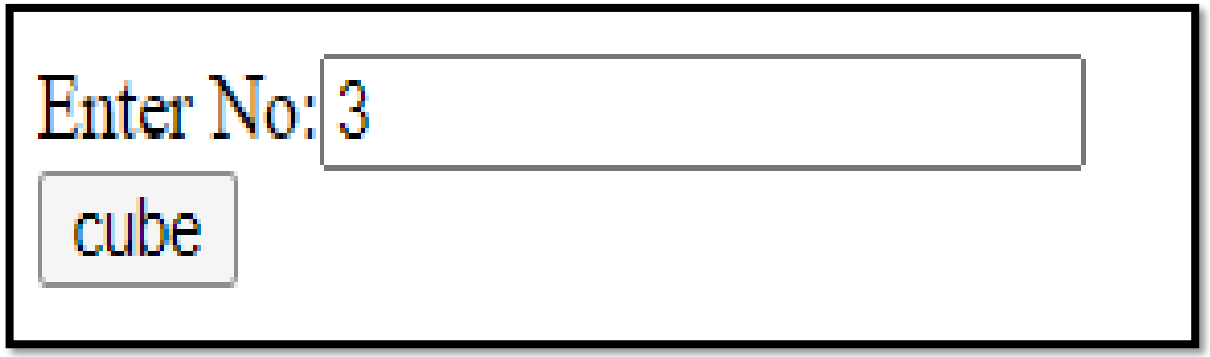




document.getElementById() method

- The **document.getElementById()** method returns the element of specified id.
- In the previous example, we have used **document.form1.name.value** to get the value of the input value.
- Instead of this, we can use **document.getElementById()** method to get value of the input text.
- But we need to define **id for the input field**.

```
<script type="text/javascript">
function getcube()
{
var number=document.getElementById("number").value;
alert(number*number*number);
}
</script>
<form>
Enter No:<input type="text" id="number" name="number"/><br/>
<input type="button" value="cube" onclick="getcube()"/>
</form>
```





document.getElementsByName() method

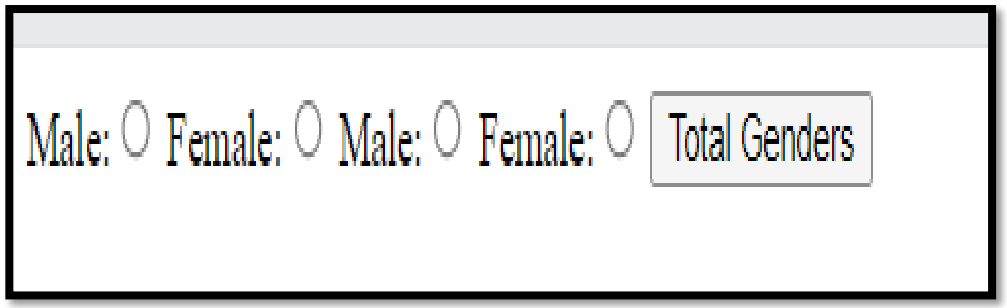
- The **document.getElementsByName()** method returns all the element of specified name.

- The syntax of the getElementsByName() method is given below:

document.getElementsByName("name")

```
<script type="text/javascript">
function totalelements()
{
var allgenders=document.getElementsByName("gender");
alert("Total Genders:"+allgenders.length);
}
</script>
<form>
Male:<input type="radio" name="gender" value="male">
Female:<input type="radio" name="gender" value="female">
Male:<input type="radio" name="gender" value="male">
Female:<input type="radio" name="gender" value="female">

<input type="button" onclick="totalelements()" value="Total Genders">
</form>
```





document.getElementsByTagName() method

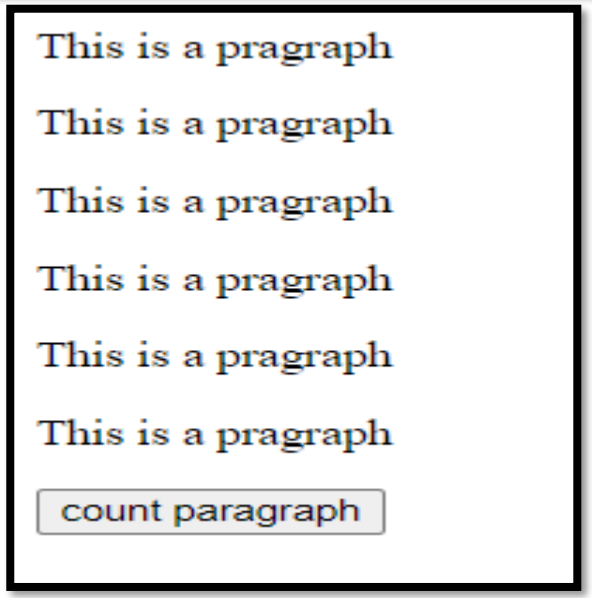
- The **document.getElementsByTagName()** method returns all the element of specified tag name.

- The syntax of the getElementsByTagName() method is given below:

document.getElementsByTagName("name")

```
<script type="text/javascript">
function countpara()
{
var totalpara=document.getElementsByTagName("p");
alert("total p tags are: "+totalpara.length);

}
</script>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<button onclick="countpara()">count paragraph</button>
```





```
<script type="text/javascript">
function counth2(){
var totalh2=document.getElementsByTagName("h2");
alert("total h2 tags are: "+totalh2.length);
}
function counth3(){
var totalh3=document.getElementsByTagName("h3");
alert("total h3 tags are: "+totalh3.length);
}
</script>
<h2>This is h2 tag</h2>
<h2>This is h2 tag</h2>
<h3>This is h3 tag</h3>
<h3>This is h3 tag</h3>
<h3>This is h3 tag</h3>
<button onclick="counth2()">count h2</button>
<button onclick="counth3()">count h3</button>
```

Browser Object Model

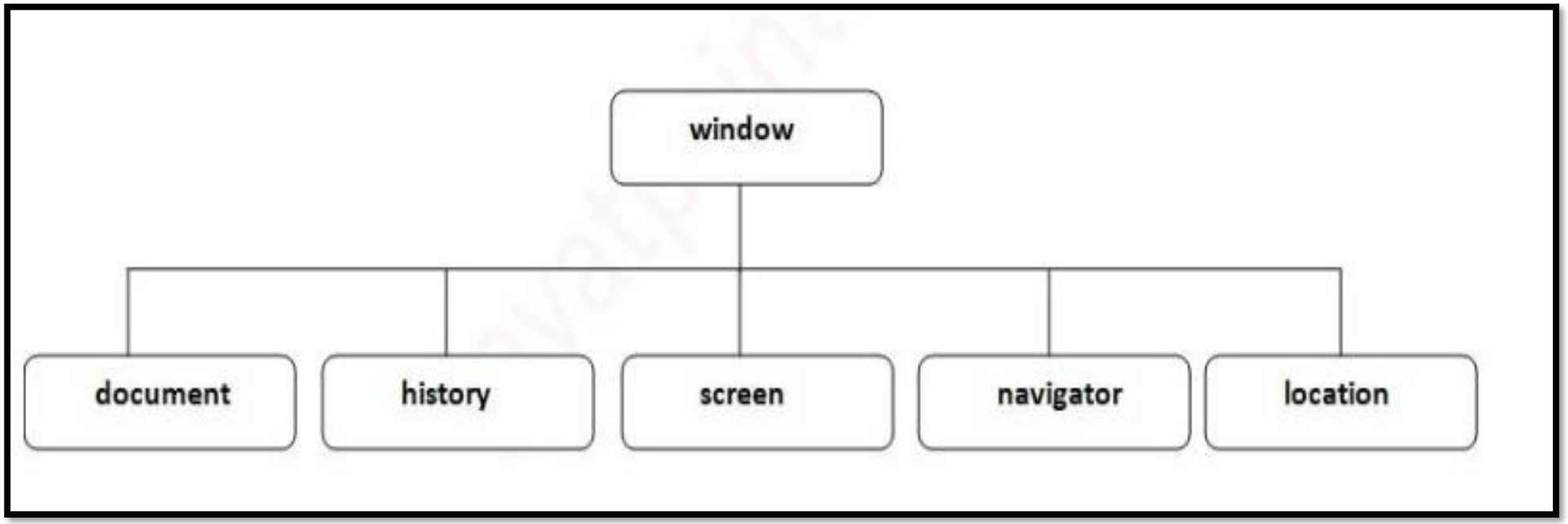
- The **Browser Object Model** (BOM) is used to interact with the browser.
- The default object of browser is window means you can call all the functions of window by specifying window or directly.
- For example:

```
window.alert("hello javatpoint");
```

is same as:

```
alert("hello javatpoint");
```

- You can use a lot of properties (other objects) defined underneath the window object like **document, history, screen, navigator, location, innerHeight, innerWidth**



Window Object

- The **window object** represents a window in browser. An object of window is created automatically by the browser.
- Window is the object of browser, **it is not the object of javascript**. The javascript objects are string, array, date etc.
- **Methods of window object**

Method	Description
alert()	displays the alert box containing message with ok button.
confirm()	displays the confirm dialog box containing message with ok and cancel button.
prompt()	displays a dialog box to get input from the user.
open()	opens the new window.
close()	closes the current window.
setTimeout()	performs action after specified time like calling function, evaluating expressions etc.

Example of open() in javascript

```
<script type="text/javascript">
function msg(){
open("http://www.Google.com");
}
</script>
<input type="button" value="Click here" onclick="msg()"/>
```



Example of setTimeout() in javascript

- It performs its task after the given milliseconds.

```
<script type="text/javascript">
function msg(){
setTimeout(
function(){
alert("Welcome to PPSU after 2 seconds")
},2000);

}
</script>

<input type="button" value="click" onclick="msg()"/>
```



JavaScript History Object

- The **JavaScript history object** represents an array of URLs visited by the user. By using this object, you can load previous, forward or any particular page.

- The history object is the window property, so it can be accessed by:

window.history

Or,

history

Property of JavaScript history object

No.	Property	Description
1	length	returns the length of the history URLs.

Methods of JavaScript history object

No.	Method	Description
1	forward()	loads the next page.
2	back()	loads the previous page.
3	go()	loads the given page number.

JavaScript Navigator Object

- The **JavaScript navigator object** is used for browser detection. It can be used to get browser information such as appName, appCodeName, userAgent etc.

- The navigator object is the window property, so it can be accessed by:

window.navigator

Or,

navigator

Property of JavaScript navigator object

No.	Property	Description
1	appName	returns the name
2	appVersion	returns the version
3	appCodeName	returns the code name
4	cookieEnabled	returns true if cookie is enabled otherwise false
5	userAgent	returns the user agent
6	language	returns the language. It is supported in Netscape and Firefox only.
7	userLanguage	returns the user language. It is supported in IE only.
8	plugins	returns the plugins. It is supported in Netscape and Firefox only.
9	systemLanguage	returns the system language. It is supported in IE only.
10	mimeTypes[]	returns the array of mime type. It is supported in Netscape and Firefox only.
11	platform	returns the platform e.g. Win32.
12	online	returns true if browser is online otherwise false.

Methods of JavaScript navigator object

No.	Method	Description
1	javaEnabled()	checks if java is enabled.
2	taintEnabled()	checks if taint is enabled. It is deprecated since JavaScript 1.2.

```
<html>
<body>
<h2>JavaScript Navigator Object</h2>
<script>
document.writeln("<br/>navigator.appCodeName: "+navigator.appCodeName);
document.writeln("<br/>navigator.appName: "+navigator.appName);
document.writeln("<br/>navigator.appVersion: "+navigator.appVersion);
document.writeln("<br/>navigator.cookieEnabled: "+navigator.cookieEnabled);
document.writeln("<br/>navigator.language: "+navigator.language);
document.writeln("<br/>navigator.userAgent: "+navigator.userAgent);
document.writeln("<br/>navigator.platform: "+navigator.platform);
document.writeln("<br/>navigator.onLine: "+navigator.onLine);

</script>
</body>
</html>
```

JavaScript Navigator Object

navigator.appCodeName: Mozilla
navigator.appName: Netscape
navigator.appVersion: 5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36
navigator.cookieEnabled: true
navigator.language: en-US
navigator.userAgent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36
navigator.platform: Win32
navigator.onLine: true

JavaScript Screen Object

- The **JavaScript screen object** holds information of browser screen. It can be used to display screen width, height, colorDepth, pixelDepth etc.
- The navigator object is the window property, so it can be accessed by:

window.screen

Or,

screen

Property of JavaScript Screen Object

No.	Property	Description
1	width	returns the width of the screen
2	height	returns the height of the screen
3	availWidth	returns the available width
4	availHeight	returns the available height
5	colorDepth	returns the color depth
6	pixelDepth	returns the pixel depth.

```
<html>
<body>
<script>
document.writeln("<br/>screen.width: "+screen.width);
document.writeln("<br/>screen.height: "+screen.height);
document.writeln("<br/>screen.availWidth: "+screen.availWidth);
document.writeln("<br/>screen.availHeight: "+screen.availHeight);
document.writeln("<br/>screen.colorDepth: "+screen.colorDepth);
document.writeln("<br/>screen.pixelDepth: "+screen.pixelDepth);
</script>
</body>
</html>
```

screen.width: 1920
screen.height: 1080
screen.availWidth: 1920
screen.availHeight: 1032
screen.colorDepth: 24
screen.pixelDepth: 24

JavaScript Type Conversion

- Converting Strings to Numbers
- Converting Numbers to Strings
- Converting Dates to Numbers
- Converting Numbers to Dates
- Converting Booleans to Numbers
- Converting Numbers to Booleans

Converting Strings to Numbers

The global method `Number()` converts a variable (or a value) into a number.

A numeric string (like "3.14") converts to a number (like 3.14).

An empty string (like "") converts to 0.

A non numeric string (like "John") converts to `NaN` (Not a Number).

```html
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Numbers</h1>
<h2>The Number() Method</h2>

<p>The Number() metod converts a variable (or value) into a number:</p>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML =
Number("3.14") + "<br>" +
Number(Math.PI) + "<br>" +
Number("    ") + "<br>" +
Number("") + "<br>" +
Number("99 88") + "<br>" +
Number("John") + "<br>";
</script>

</body>
</html>
```

JavaScript Numbers

The Number() Method

The Number() metod converts a variable (or value) into a number:

3.14
3.141592653589793
0
0
NaN
NaN

Number Methods

Method	Description
Number()	Returns a number, converted from its argument
parseFloat()	Parses a string and returns a floating point number
parseInt()	Parses a string and returns an integer

The Unary + Operator

- The **unary + operator** can be used to convert a variable to a number:

```
<!DOCTYPE html>
<html>
<body>

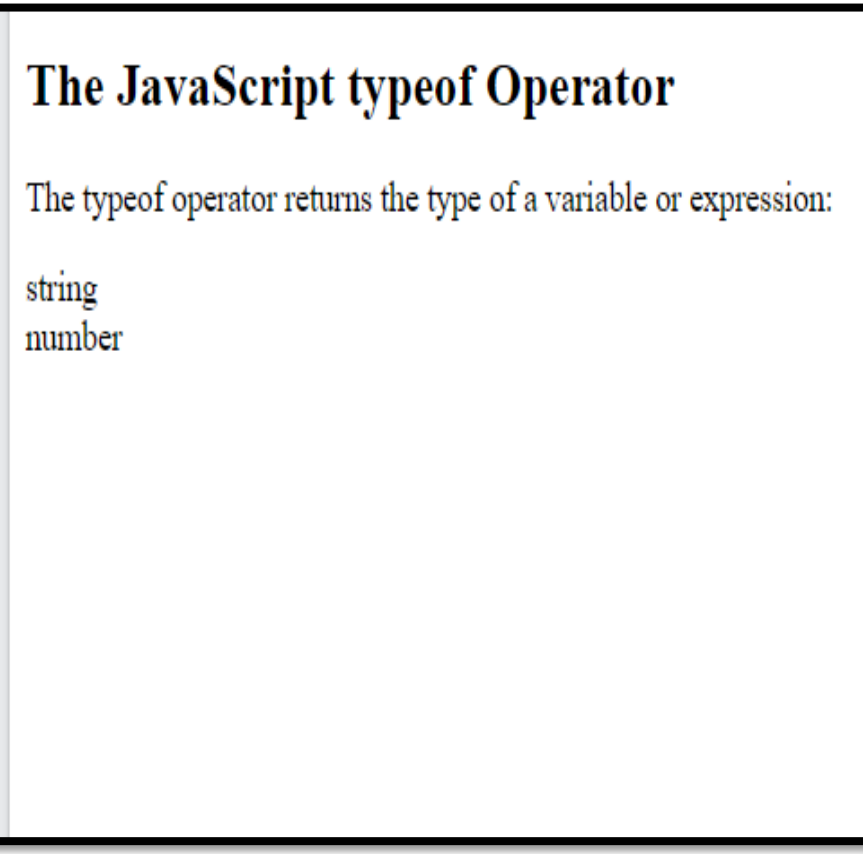
<h2>The JavaScript typeof Operator</h2>

<p>The typeof operator returns the type of a variable or expression:</p>

<p id="demo"></p>

<script>
let y = "5";
let x = + y;
document.getElementById("demo").innerHTML = typeof y + "<br>" + typeof x;
</script>

</body>
</html>
```



Converting Numbers to Strings

- The global method String() can convert numbers to strings.

```
String(x)         // returns a string from a number variable x
String(123)       // returns a string from a number literal 123
String(100 + 23)  // returns a string from a number from an expression
```

Methods of document object

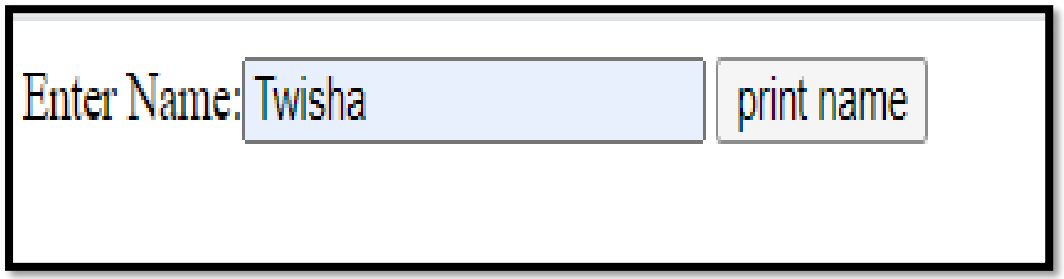
Method	Description
write("string")	writes the given string on the doucment.
writeln("string")	writes the given string on the doucment with newline character at the end.
getElementById()	returns the element having the given id value.
getElementsByName()	returns all the elements having the given name value.
getElementsByTagName()	returns all the elements having the given tag name.
getElementsByClassName()	returns all the elements having the given class name.

Accessing field value by document object

- **document.form1.name.value** is used to get the value of name field.
- Here, **document** is the root element that represents the html document.
- **form1** is the name of the form.
- **name** is the attribute name of the input text.
- **value** is the property, that returns the value of the input text.

```
<script type="text/javascript">
function printvalue()
{
var name=document.form1.name.value;
alert("Welcome: "+name);
}
</script>

<form name="form1">
Enter Name:<input type="text" name="name"/>
<input type="button" onclick="printvalue()" value="print name"/>
</form>
```

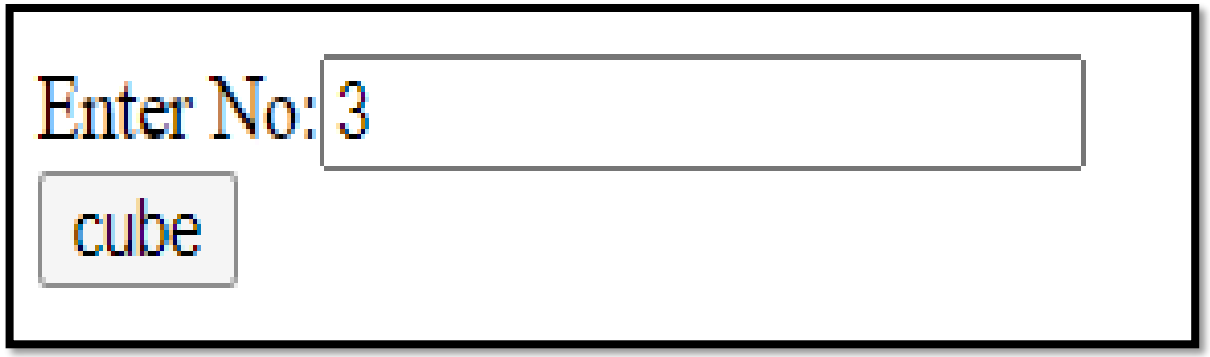




document.getElementById() method

- The **document.getElementById()** method returns the element of specified id.
- In the previous example, we have used **document.form1.name.value** to get the value of the input value.
- Instead of this, we can use **document.getElementById()** method to get value of the input text.
- But we need to define **id for the input field**.

```
<script type="text/javascript">
function getcube()
{
var number=document.getElementById("number").value;
alert(number*number*number);
}
</script>
<form>
Enter No:<input type="text" id="number" name="number"/><br/>
<input type="button" value="cube" onclick="getcube()"/>
</form>
```





document.getElementsByName() method

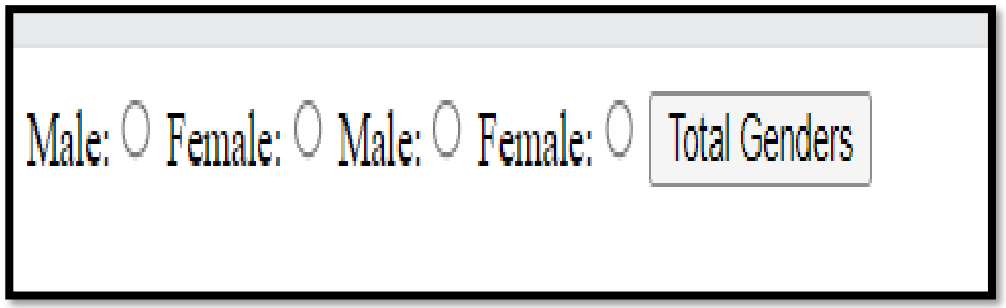
- The **document.getElementsByName()** method returns all the element of specified name.

- The syntax of the getElementsByName() method is given below:

document.getElementsByName("name")

```
<script type="text/javascript">
function totalelements()
{
var allgenders=document.getElementsByName("gender");
alert("Total Genders:"+allgenders.length);
}
</script>
<form>
Male:<input type="radio" name="gender" value="male">
Female:<input type="radio" name="gender" value="female">
Male:<input type="radio" name="gender" value="male">
Female:<input type="radio" name="gender" value="female">

<input type="button" onclick="totalelements()" value="Total Genders">
</form>
```





document.getElementsByTagName() method

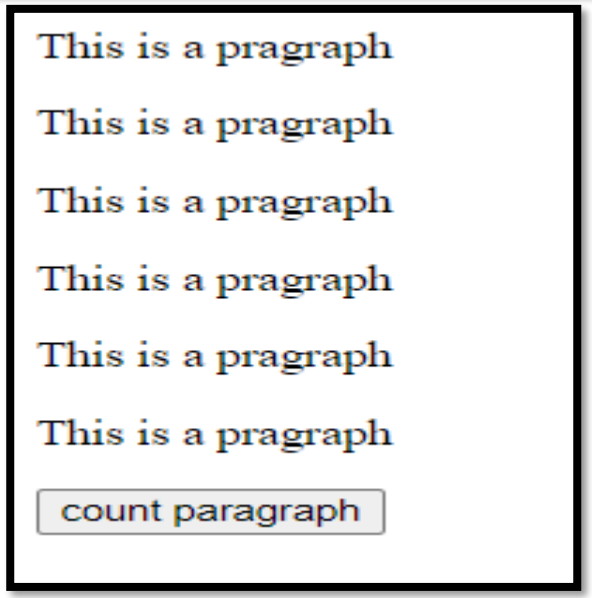
- The **document.getElementsByTagName()** method returns all the element of specified tag name.

- The syntax of the getElementsByTagName() method is given below:

document.getElementsByTagName("name")

```
<script type="text/javascript">
function countpara()
{
var totalpara=document.getElementsByTagName("p");
alert("total p tags are: "+totalpara.length);

}
</script>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<p>This is a pragraph</p>
<button onclick="countpara()">count paragraph</button>
```





```
<script type="text/javascript">
function counth2(){
var totalh2=document.getElementsByTagName("h2");
alert("total h2 tags are: "+totalh2.length);
}
function counth3(){
var totalh3=document.getElementsByTagName("h3");
alert("total h3 tags are: "+totalh3.length);
}
</script>
<h2>This is h2 tag</h2>
<h2>This is h2 tag</h2>
<h3>This is h3 tag</h3>
<h3>This is h3 tag</h3>
<h3>This is h3 tag</h3>
<button onclick="counth2()">count h2</button>
<button onclick="counth3()">count h3</button>
```

Browser Object Model

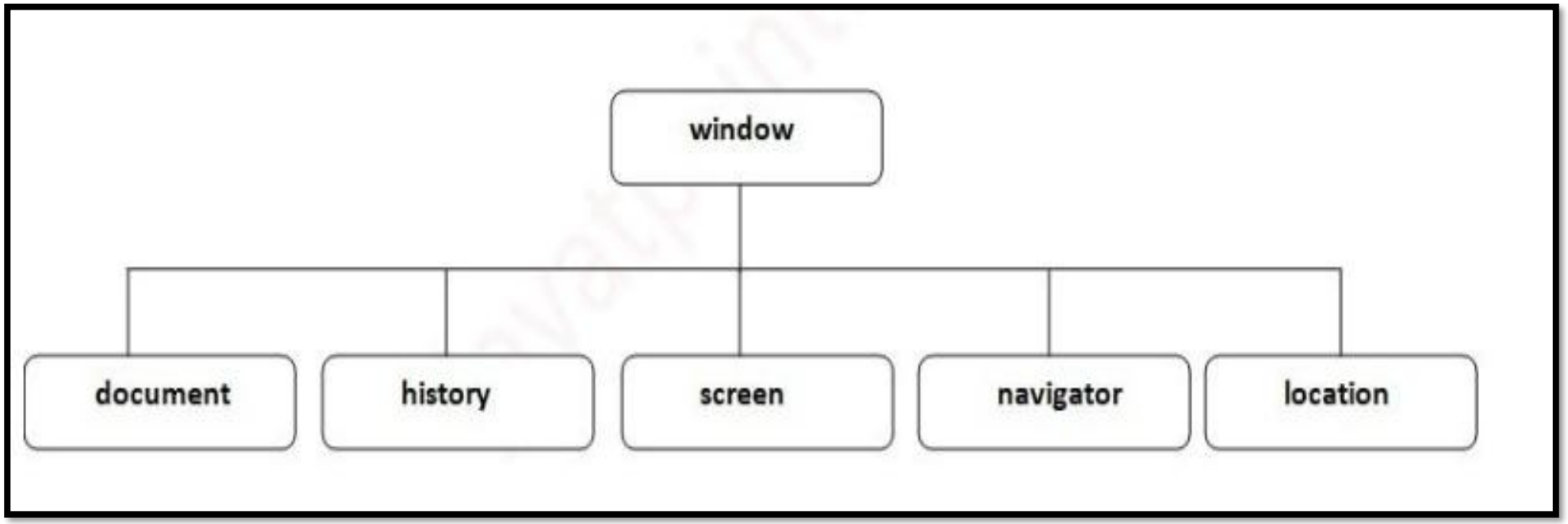
- The **Browser Object Model** (BOM) is used to interact with the browser.
- The default object of browser is window means you can call all the functions of window by specifying window or directly.
- For example:

```
window.alert("hello javatpoint");
```

is same as:

```
alert("hello javatpoint");
```

- You can use a lot of properties (other objects) defined underneath the window object like **document, history, screen, navigator, location, innerHeight, innerWidth**



Window Object

- The **window object** represents a window in browser. An object of window is created automatically by the browser.
- Window is the object of browser, **it is not the object of javascript**. The javascript objects are string, array, date etc.
- **Methods of window object**

Method	Description
alert()	displays the alert box containing message with ok button.
confirm()	displays the confirm dialog box containing message with ok and cancel button.
prompt()	displays a dialog box to get input from the user.
open()	opens the new window.
close()	closes the current window.
setTimeout()	performs action after specified time like calling function, evaluating expressions etc.

Example of open() in javascript

```
<script type="text/javascript">
function msg(){
open("http://www.Google.com");
}
</script>
<input type="button" value="Click here" onclick="msg()"/>
```



Example of setTimeout() in javascript

- It performs its task after the given milliseconds.

```
<script type="text/javascript">
function msg(){
setTimeout(
function(){
alert("Welcome to PPSU after 2 seconds")
},2000);

}
</script>

<input type="button" value="click" onclick="msg()"/>
```



JavaScript History Object

- The **JavaScript history object** represents an array of URLs visited by the user. By using this object, you can load previous, forward or any particular page.

- The history object is the window property, so it can be accessed by:

window.history

Or,

history

Property of JavaScript history object

No.	Property	Description
1	length	returns the length of the history URLs.

Methods of JavaScript history object

No.	Method	Description
1	forward()	loads the next page.
2	back()	loads the previous page.
3	go()	loads the given page number.

JavaScript Navigator Object

- The **JavaScript navigator object** is used for browser detection. It can be used to get browser information such as appName, appCodeName, userAgent etc.

- The navigator object is the window property, so it can be accessed by:

window.navigator

Or,

navigator

Property of JavaScript navigator object

No.	Property	Description
1	appName	returns the name
2	appVersion	returns the version
3	appCodeName	returns the code name
4	cookieEnabled	returns true if cookie is enabled otherwise false
5	userAgent	returns the user agent
6	language	returns the language. It is supported in Netscape and Firefox only.
7	userLanguage	returns the user language. It is supported in IE only.
8	plugins	returns the plugins. It is supported in Netscape and Firefox only.
9	systemLanguage	returns the system language. It is supported in IE only.
10	mimeTypes[]	returns the array of mime type. It is supported in Netscape and Firefox only.
11	platform	returns the platform e.g. Win32.
12	online	returns true if browser is online otherwise false.

Methods of JavaScript navigator object

No.	Method	Description
1	javaEnabled()	checks if java is enabled.
2	taintEnabled()	checks if taint is enabled. It is deprecated since JavaScript 1.2.

```
<html>
<body>
<h2>JavaScript Navigator Object</h2>
<script>
document.writeln("<br/>navigator.appCodeName: "+navigator.appCodeName);
document.writeln("<br/>navigator.appName: "+navigator.appName);
document.writeln("<br/>navigator.appVersion: "+navigator.appVersion);
document.writeln("<br/>navigator.cookieEnabled: "+navigator.cookieEnabled);
document.writeln("<br/>navigator.language: "+navigator.language);
document.writeln("<br/>navigator.userAgent: "+navigator.userAgent);
document.writeln("<br/>navigator.platform: "+navigator.platform);
document.writeln("<br/>navigator.onLine: "+navigator.onLine);

</script>
</body>
</html>
```

JavaScript Navigator Object

navigator.appCodeName: Mozilla
navigator.appName: Netscape
navigator.appVersion: 5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36
navigator.cookieEnabled: true
navigator.language: en-US
navigator.userAgent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36
navigator.platform: Win32
navigator.onLine: true

JavaScript Screen Object

- The **JavaScript screen object** holds information of browser screen. It can be used to display screen width, height, colorDepth, pixelDepth etc.
- The navigator object is the window property, so it can be accessed by:

window.screen

Or,

screen

Property of JavaScript Screen Object

No.	Property	Description
1	width	returns the width of the screen
2	height	returns the height of the screen
3	availWidth	returns the available width
4	availHeight	returns the available height
5	colorDepth	returns the color depth
6	pixelDepth	returns the pixel depth.

```
<html>
<body>
<script>
document.writeln("<br/>screen.width: "+screen.width);
document.writeln("<br/>screen.height: "+screen.height);
document.writeln("<br/>screen.availWidth: "+screen.availWidth);
document.writeln("<br/>screen.availHeight: "+screen.availHeight);
document.writeln("<br/>screen.colorDepth: "+screen.colorDepth);
document.writeln("<br/>screen.pixelDepth: "+screen.pixelDepth);
</script>
</body>
</html>
```

screen.width: 1920
screen.height: 1080
screen.availWidth: 1920
screen.availHeight: 1032
screen.colorDepth: 24
screen.pixelDepth: 24